Securex: A Comprehensive CCTV & Security Solution System

by

Shanawaz Sakib
ID: 11220320916

CSE 4100
Field Work / Industrial Training

Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna 9208, Bangladesh

August, 2026

Declaration

This is to certify that the Field Work / Industrial Training work entitled "Securex: A Comprehensive CCTV & Security Solution System" has been carried out by Md. Nazmus Shakib in the Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, Khulna 9208, Bangladesh. The above Field Work / Industrial Training work or any part of this work has not been submitted anywhere for the award of any degree or diploma.

Signature of the Supervisor **Signature of the Student**

Md. Zahid Hasan Rabbi
Lecturer
Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna 9208, Bangladesh

Shanawaz Sakib
11220320916
7A

Acknowledgement

All praise and gratitude belong to Almighty Allah for granting me the strength, patience, and knowledge to complete this project successfully.

I would like to express my sincere gratitude and heartfelt appreciation to my supervisor, Md. Zahid Hasan Rabbi, Lecturer, Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, for his invaluable guidance, continuous encouragement, constructive criticism, and generous support throughout the course of this project. His insightful suggestions and scholarly mentorship were indispensable in shaping the direction and quality of this work.

I extend my sincere thanks to the Faculty Members and the Chairman of the Department of Computer Science and Engineering, Northern University of Business and Technology Khulna, for providing the necessary academic environment and infrastructure that facilitated this project.

I am also deeply indebted to my family for their unwavering moral support and encouragement throughout my academic journey. Their sacrifice and belief in my abilities have been a constant source of motivation.

Finally, I acknowledge the contributions of the open source community, especially the developers behind the Laravel framework, the Socialite OAuth package, and all related technologies, without whose foundational work this project would not have been possible.

Shanawaz Sakib

August, 2026

Contents

Page

Title Page ..... **i**

Declaration ..... **ii**

Acknowledgement ..... **iii**

Contents ..... **iv**

List of Figures ..... **vi**

List of Abbreviations ..... **vii**

CHAPTER I INTRODUCTION **1**

1.1 Introduction 1

1.2 Motivation 2

1.3 Objectives and Specific Aims 2

1.4 Organization of the Report 3

CHAPTER II PROCEDURE / METHODOLOGY **4**

2.1 System Architecture 4

2.2 Core Modules 7

2.2.1 Frontend Module 7

2.2.2 Authentication Module 7

2.2.3 Booking Logic and Time Slot Validation 8

2.2.4 Admin Workspace 11

2.3 Technical Challenge: Routing Refactoring and Resolution 12

CHAPTER III RESULTS AND DISCUSSION **14**

3.1 System Implementation Results 14

3.2 User Interface Screenshots 15

CHAPTER IV CONCLUSIONS AND RECOMMENDATIONS 24

4.1 Conclusions 24

4.2 Recommendations for Future Work 25

References 27

List of Figures

Figure No.	Description	Page
2.1	Securex Database ER Diagram	5
2.2	Overall Securex System Flowchart	6
2.3	User Booking Process Flowchart	10
2.4	Admin Booking Approval Flowchart	12
3.1	Securex Home Page	15
3.2	Services Catalog Page	15
3.3	Service Booking Form	16
3.4	Login Page with Google OAuth	16
3.5	User Registration Page	17
3.6	Google OAuth Consent Screen	17
3.7	My Bookings Page	18
3.8	Admin Dashboard	18
3.9	Admin Service List	19
3.10	Admin Edit Service Form	19
3.11	Admin Add Service Form	20
3.12	Admin User Management	20
3.13	Admin Booking Requests	21
3.14	My Bookings Status and Actions	21
3.15	Service Invoice	22
3.16	Time Slot Conflict Validation	22
3.17	Automated Booking Email	23

List of Abbreviations

API	Application Programming Interface
CCTV	Closed Circuit Television
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DB	Database
ERD	Entity Relationship Diagram
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
MVC	Model View Controller
NUBTK	Northern University of Business and Technology Khulna
OAuth	Open Authorization
ORM	Object Relational Mapping
PDF	Portable Document Format
PHP	PHP: Hypertext Preprocessor
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

CHAPTER I

Introduction

1.1 Introduction

In contemporary society, the assurance of physical security has become a paramount concern for residential, commercial, and institutional establishments alike. As urbanization continues to accelerate and security threats become increasingly sophisticated, the demand for reliable, professionally managed security solutions has grown exponentially. Traditional methods of manually coordinating security service bookings through telephone calls, handwritten ledgers, or unstructured email correspondence are fraught with inefficiencies, susceptible to human error, and fundamentally incapable of providing the real time visibility that both service providers and clients require.

Securex is a full featured, web based Security Service Management System developed using the PHP Laravel framework [1], [2] to address these challenges in a holistic and technologically sound manner. The system digitizes the complete operational workflow of a professional security service provider from the public facing display of available services to the entire lifecycle of a client's booking request, culminating in automated notifications and invoice delivery.

The platform distinguishes itself through the implementation of an intelligent time slot conflict detection engine, a dual mode authentication system supporting both traditional credential based login and Google OAuth 2.0 [3], a dynamic service catalog fully manageable by an administrator, and an automated email pipeline that dispatches booking status updates and PDF formatted service invoices directly to the client's registered email address.

This report documents the conception, design, implementation, and outcomes of the Securex project, developed as part of the Field Work / Industrial Training component of the Bachelor of Science in Computer Science and Engineering program at Northern University of Business and Technology Khulna.

1.2 Motivation

The impetus for developing Securex arose from a critical observation of the operational gaps prevalent in the security service industry. Many security solution providers, particularly small to medium enterprises, continue to rely on fragmented, manual, and paper based booking systems. Such systems are characterized by a complete absence of real time booking validation, leading to frequent double booking scenarios where two clients inadvertently secure the same technician slot. This not only erodes client trust but results in significant operational and reputational costs for the service provider.

Furthermore, clients are typically left without any automated acknowledgement or structured record of their service engagements. The absence of a centralized digital portal means that clients cannot independently review the history or current status of their bookings, and administrators have no efficient mechanism for tracking, prioritizing, or updating service requests without resorting to manual cross referencing.

From a software engineering perspective, the project also served as a practical vehicle for developing and demonstrating applied competencies in the Model View Controller (MVC) architectural pattern, relational database design and interaction via Eloquent ORM, third party API integration through the Laravel Socialite package, and automated email dispatch and dynamic PDF generation. The project provided an opportunity to confront and resolve real world implementation challenges, notably a complex routing configuration bug, thereby reinforcing problem solving skills in a production-adjacent environment [1], [10].

1.3 Objectives and Specific Aims

The primary objective of the Securex project is to develop a robust, scalable, and user centric web application that streamlines the end to end operations of a security service provider. The specific aims that guided the design and development of the system are enumerated as follows:

1. To design and implement a dynamic, database driven public facing website showcasing the company's complete range of security services with up to date pricing and imagery.

2. To develop a secure, dual mode user authentication system incorporating standard credential based registration and login alongside Google OAuth 2.0 single sign on capability.

3. To implement an intelligent booking engine that enforces strict time slot validation, prevents double bookings, and provides contextual guidance by suggesting available alternative slots.

4. To build a comprehensive administrative dashboard enabling full CRUD (Create, Read, Update, Delete) operations on the service catalog, user management, and booking lifecycle control.

5. To automate the client communication pipeline by triggering dynamic email notifications and PDF invoice generation upon key booking status changes.

6. To develop and deploy the application adhering to Laravel best practices, including clean RESTful routing, middleware based access control, and MVC separation of concerns [1], [4].

1.4 Organization of the Report

This report is organized into four chapters. Chapter I introduces the Securex project, including its background, motivation, objectives, and specific aims. Chapter II describes the development methodology, system architecture, core modules, booking and administrative workflows, and the major technical challenge encountered during implementation. Chapter III presents the implementation results and user interface screenshots. Finally, Chapter IV summarizes the conclusions and provides recommendations for future improvement.

CHAPTER II

Procedure / Methodology

2.1 System Architecture

Securex is architected in strict accordance with the Model View Controller (MVC) design pattern, as natively enforced by the PHP Laravel framework [1], [4]. This architectural decision provides a clean and maintainable separation of concerns across three distinct logical layers, each with well defined responsibilities.

The Model layer encapsulates the application's data logic and interactions with the MySQL relational database [6]. Eloquent ORM [1], Laravel's built in active record implementation, is employed for all database operations, providing an expressive and object oriented interface to the underlying relational data. The principal models include the User model (managing client and administrator records), the Service model (representing the security service catalog), and the Booking model (tracking the full lifecycle of every client booking request). Supporting models include Sessions and Password Reset Tokens for authentication state management, and Contact Messages for visitor inquiries submitted through the public website.

Figure 2.1 presents the Entity Relationship (ER) Diagram of the Securex database schema. The schema is deliberately normalized around one to many relationships: a single user may hold many session records and submit many bookings, while a single service may be referenced by many booking and contact message records. This design avoids data duplication, enforces referential integrity through foreign keys (e.g., user_id in the bookings and sessions tables, service_id in the bookings and contact_messages tables), and allows the application to efficiently retrieve related records through Eloquent's relationship methods.



Fig. 2.1: Entity Relationship Diagram of the Securex database. The users table connects with sessions and bookings; services connects with bookings and contact_messages; and password_reset_tokens supports account recovery. The key links show how related records are organized and retrieved without duplicating data.

The View layer is composed of dynamic Blade templating engine [1] files. Blade allows for the elegant composition of reusable HTML components with embedded PHP directives, enabling the generation of personalized and data driven web pages. The public facing views include the home page, about us page, service catalog, and booking form, while the administrative views constitute the secured control panel.

The Controller layer acts as the orchestrating intermediary, receiving incoming HTTP requests, invoking the appropriate Model logic for data retrieval or manipulation, and returning the appropriate View response. Key controllers include the BookingController (which houses the time slot validation logic), the ServiceController (managing the admin's CRUD operations on the service catalog), and the AuthController (managing authentication flows). The application employs Laravel's named routing system [1], [4], grouping public routes separately from routes protected by the auth and admin middleware stacks, ensuring that unauthorized access to sensitive endpoints is systematically rejected.

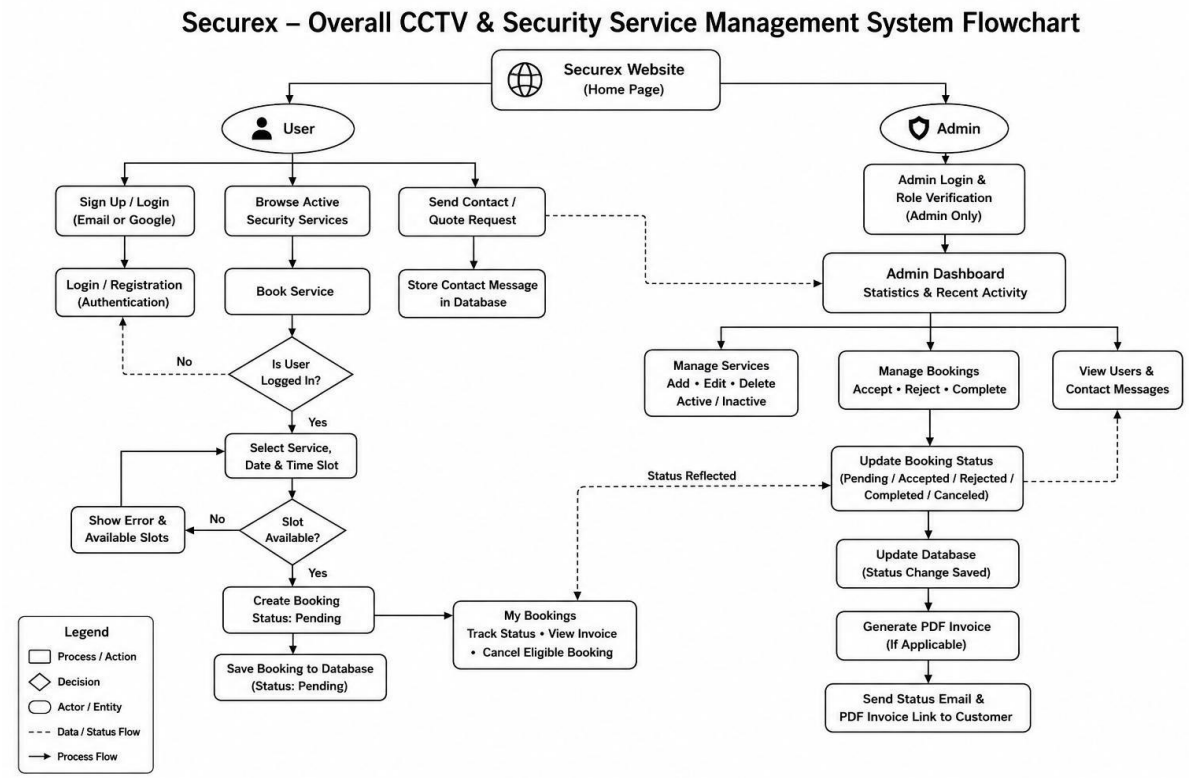


Fig. 2.2: Overall System Flowchart of Securex. The diagram summarizes the six major paths - service browsing, registration/login, booking, My Bookings, contact/inquiry, and the admin workspace - together with their decisions, database updates, email notifications, and end states.

Overall System Flowchart Explanation

Figure 2.2 presents the complete operational process of the Securex CCTV and Security Solution System. The process begins when a visitor opens the Securex website and reaches the home page. From there, users can browse available security services, view service information, register a new account, or log in using an email and password or Google authentication. After successful login, a user can open the booking form, select a preferred service, booking date, and time slot, and submit the required information. Before confirming the booking, the system validates all entered data and checks whether the selected time slot is available. If the chosen slot is unavailable, the system displays an appropriate message and suggests alternative available time slots. Once the booking is successfully submitted, it is stored in the database with a Pending status.

Through the My Bookings section, users can view their booking history, monitor the current booking status, cancel eligible bookings, and access invoices for completed services. Visitors and registered users can also submit inquiries through the contact form, which are stored in the database for administrative review.

The administrator accesses the system through the login page and is redirected to the admin dashboard after successful authentication. From the dashboard, the administrator can manage services, view registered users and contact messages, and process booking requests. Pending bookings can be accepted or rejected, while accepted bookings can later be marked as completed. After every important status change, the system updates the database and automatically sends an email notification to the user. Therefore, the flowchart clearly demonstrates how Securex provides an organized, reliable, and user-friendly platform for managing security service bookings and administrative activities.

2.2 Core Modules

2.2.1 Frontend Module

The public facing frontend of Securex is designed to serve as a comprehensive marketing and information portal for the security service company. The Home page features a prominent hero section with a compelling call to action, followed by an "Our Services" grid section dynamically populated from the database, displaying service titles, images, and pricing. This ensures that any service added or modified by an administrator is immediately reflected on the public facing page without requiring any manual code changes.

The About Us page presents the company's value proposition through organized information grids highlighting core service pillars such as Home Security, Access Control, and 24/7 Support. The dedicated Service page provides a more exhaustive catalog view, enabling prospective clients to browse the full range of available security solutions with their associated pricing before initiating a booking. The navigation structure is fully dynamic, and the booking section is accessible via multiple call to action links throughout the site, ensuring a seamless user journey.

2.2.2 Authentication Module

Access to the booking functionality of Securex is strictly gated behind a mandatory authentication requirement. The authentication module was developed using Laravel's built in authentication scaffolding, significantly extended with custom logic to support two distinct methods of user onboarding.

The first method is the traditional credential based system: a user navigates to the registration page, provides their full name, email address, phone number, and a password, and upon successful validation, their account is created and persisted in the database with an assigned role of 'user'. Subsequent login is performed via the standard email and password combination.

The second method leverages Google OAuth 2.0 [3] via the Laravel Socialite package. When a user selects the 'Sign in with Google' option on the login page, they are redirected to Google's consent screen where they authorize the application to access their name and email address. Upon authorization, the Google API redirects the user back to the application with an authentication token. The application's callback controller method retrieves the user's profile information from Google, and either creates a new user record

in the database (for first time Google sign ins) or retrieves the existing corresponding record (for returning users), before establishing an authenticated session. The administrator account is distinguished by a reserved role designation in the database and is the sole user type with access to the admin panel, enforced through a dedicated middleware gate.

2.2.3 Booking Logic and Time Slot Validation

The service booking engine constitutes the functional core of the Securex platform and contains the most complex business logic of the application. When an authenticated user navigates to the booking page, the form presents a dynamically populated dropdown of available services sourced from the database, a date picker, and a time slot selector offering standardized appointment times.

Upon form submission, the BookingController's store method executes a multi step validation process. First, standard Laravel form validation rules verify that all required fields are present and correctly formatted. Critically, the controller then executes a database query to check whether any existing, non cancelled booking record already occupies the exact combination of the selected service, date, and time slot. This is the time slot conflict detection mechanism.

If a conflict is detected, the system does not simply reject the request with a generic error message. Instead, it performs a secondary, follow up query to retrieve all time slots for the same date and service that are not yet booked. This list of available alternatives is then formatted and incorporated into a descriptive error message for example: "This slot is already booked. Please choose another time. Available slots for this date: 09:00 AM, 11:00 AM, 05:00 PM" and returned to the booking form view. This user centric validation approach significantly improves the user experience by providing actionable guidance rather than a dead end. If no conflict is detected, a new Booking record is created in the database with an initial status of 'Pending', and the user is notified of the successful submission [6].

Figure 2.3 illustrates the complete end to end user booking process, from the initial homepage visit and login check, through the time slot availability check and booking submission, to the final tracking of the booking status Pending, Accepted, Completed, or Canceled on the client's My Bookings page.

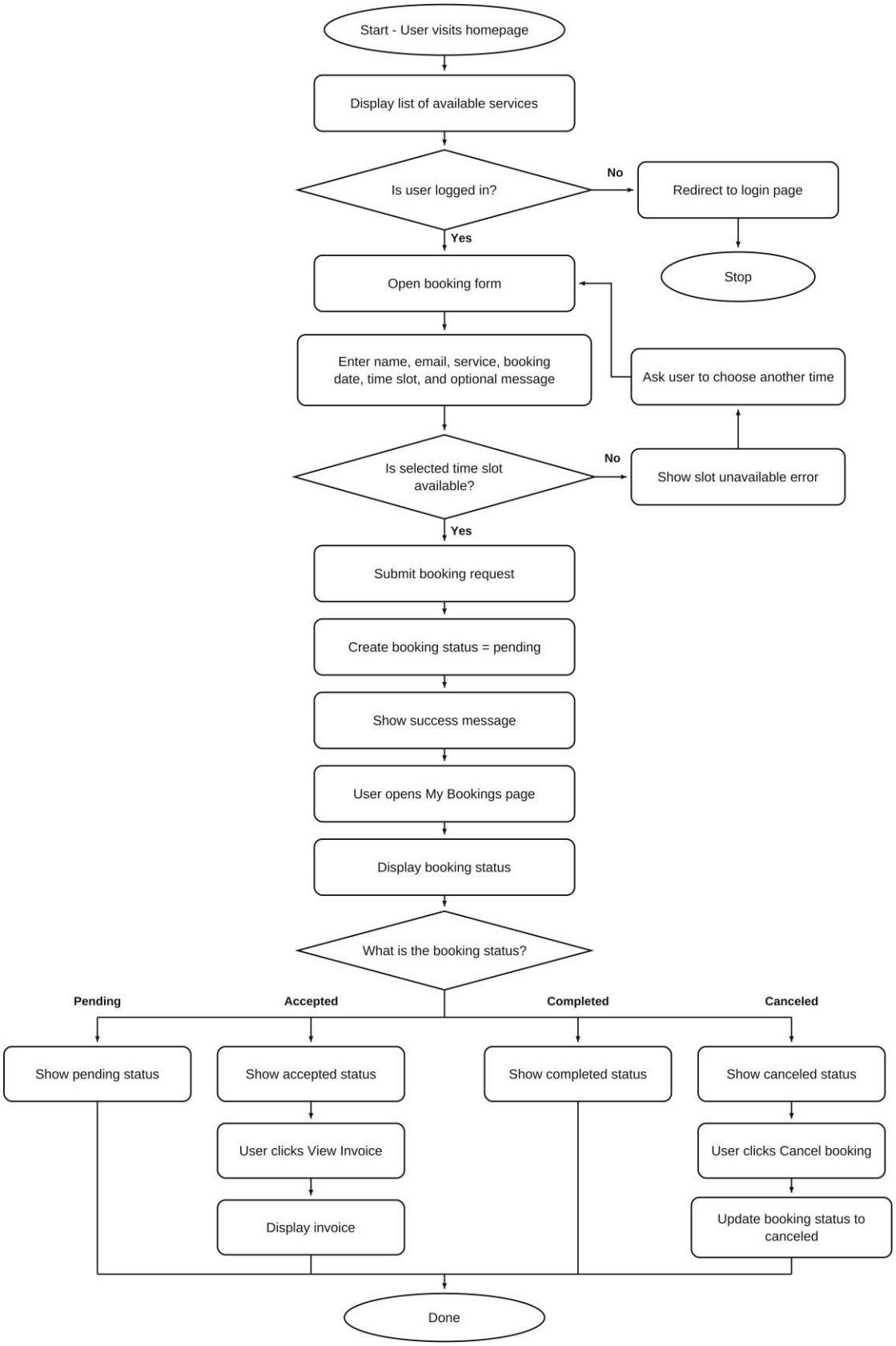


Fig. 2.3: User Booking Process Flowchart. The flow begins with service discovery, checks whether the user is authenticated, validates the selected date and time slot, creates a Pending booking when the slot is free, and then supports status tracking, permitted cancellation, and invoice viewing.

2.2.4 Admin Workspace

The administrative workspace, accessible exclusively to users with the 'admin' role designation, provides a comprehensive suite of management tools. The Admin Dashboard presents a high level operational overview, including key metrics such as total registered users, the count of active services in the catalog, the total number of booking requests received, and the number of messages pending review. A 'Today's Focus' panel highlights urgent action items, specifically the count of booking requests awaiting a decision.

The Service Management module enables the administrator to perform full CRUD operations on the service catalog. Services are listed in a tabular format displaying the service image, title, price, description, and current status (Active or Inactive). The 'Active on homepage' toggle in the edit form provides granular control over which services are visible to the public without requiring deletion. Administrators can add new services with an image URL, title, description, and price, or modify and delete existing entries.

The Booking Requests module provides the administrator with a comprehensive list of all submitted bookings. For each booking, the admin can view the client's name, email, the service requested, the price, the scheduled date and time, and the current status. Pending requests present the administrator with 'Accept' and 'Reject' action buttons. Accepted requests present a 'Complete' button to mark the service as delivered. When an admin changes a booking's status to 'Accepted' or 'Completed', the system automatically triggers the notification pipeline, dispatching an email and generating an invoice for the client.

Figure 2.4 depicts the administrator's decision workflow for processing booking requests: a Pending booking may be Accepted or Rejected, and an Accepted booking may subsequently be marked Completed; in every case a status change notification email is dispatched to the client automatically, whereas Rejected or Completed bookings no longer permit further status changes.

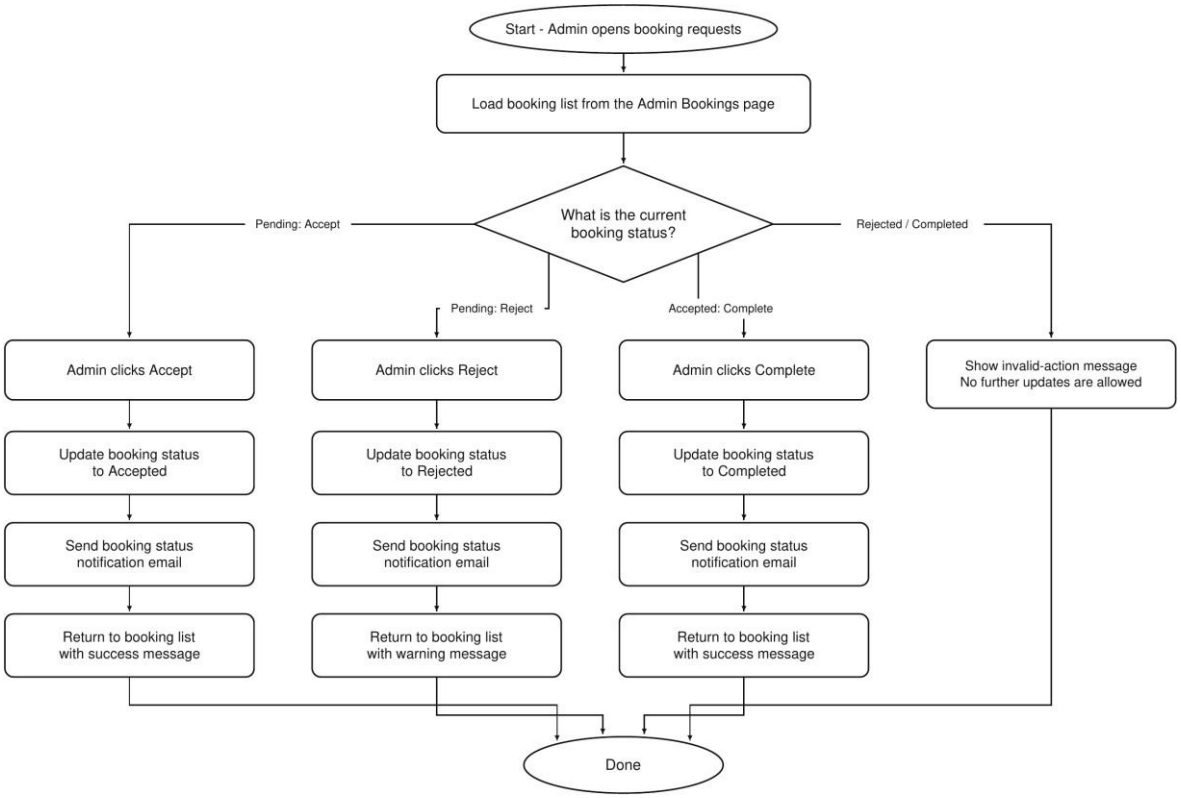


Fig. 2.4: Admin Booking Approval Flowchart. Pending requests can be Accepted or Rejected, Accepted requests can be marked Completed, and every valid status change updates the database and sends an email notification. Rejected and Completed states prevent further modification.

The User Management module presents a directory of all registered users, displaying their name, email, phone number, assigned role, and registration date, providing the administrator with a complete overview of the platform's user base.

2.3 Technical Challenge: Routing Refactoring and Resolution

During the development and integration phase of the project, a significant technical complication was encountered pertaining to the application's URL routing behavior. Navigation hyperlinks within the application's Blade templates were found to be incorrectly resolving to static file paths appended with the .html extension (e.g., navigating to /about.html instead of the intended dynamic route /about). This behavior

indicated that the navigation links were being constructed as static HTML anchor tags with hardcoded relative file paths, rather than utilizing Laravel's dynamic URL generation helpers.

The root cause was identified as the use of bare <a href="about.html"> style links in the Blade templates, a pattern characteristic of static HTML websites but incompatible with a server side rendered, framework managed application. This caused the browser to resolve

the links as file system paths relative to the current URL rather than as named routes registered in the application's routing table.

The resolution involved a systematic refactoring of all navigation links across the affected Blade template files. All hardcoded static .html references were replaced with Laravel's native route() and url() helper functions, which dynamically generate the correct, clean URLs at render time based on the application's defined named routes. For example, href="about.html" was refactored to href="{{ route('about') }}". Following this refactoring, all navigation links correctly resolved to their intended dynamic controller endpoints, producing clean, SEO friendly, and application managed URLs. This exercise reinforced the importance of adhering to framework specific conventions and the benefits of using named routes for maintainability.

CHAPTER III

Results and Discussion

3.1 System Implementation Results

The development of the Securex platform was completed successfully, with all core modules implemented, tested, and validated in a local development environment running on Laravel's built in development server (localhost:8000) with a MySQL database backend. The results confirm that the system satisfactorily fulfills all the objectives stipulated in Chapter I.

The intelligent time slot validation engine reliably detected and blocked conflicting bookings across multiple test scenarios, returning accurate lists of available alternative slots. The Google OAuth integration functioned correctly end to end, from the initial authorization redirect to the successful user session establishment. The administrative dashboard provided comprehensive control over all system entities, and the automated email notification system successfully dispatched booking updates and invoice links to the registered client email addresses. The following subsection presents annotated screenshots documenting the implemented interfaces.

3.2 User Interface Screenshots

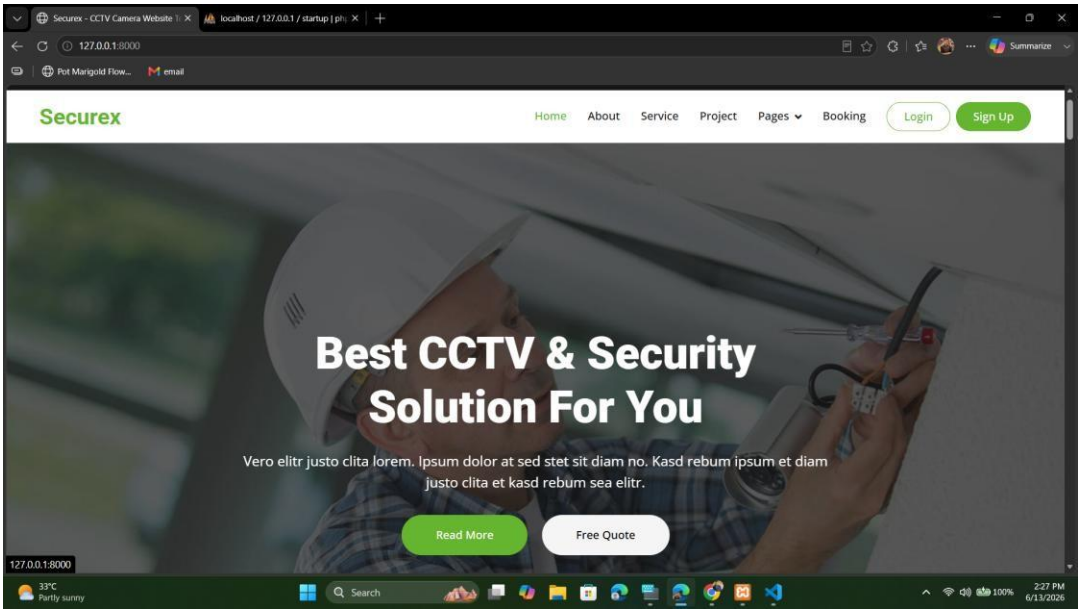


Fig. 3.1: Securex Home Page. The interface combines the primary navigation menu, a security-focused hero banner, and prominent action buttons that guide visitors to explore the service catalog or request a quotation.

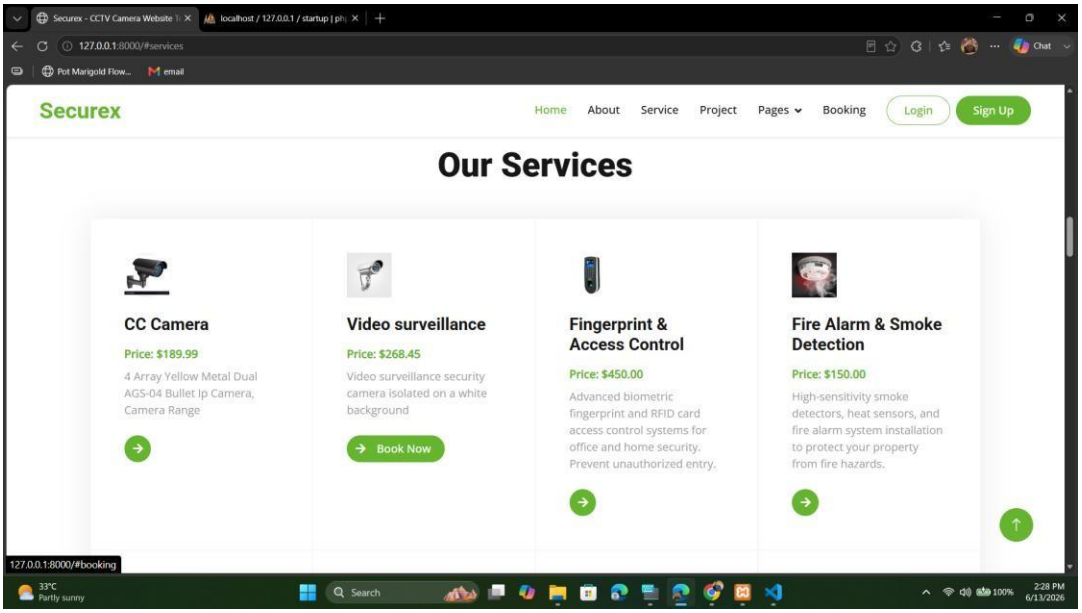


Fig. 3.2: Services Catalog Page. Each service card presents an image, service title, price, short description, and booking control, allowing customers to compare available security solutions before continuing.

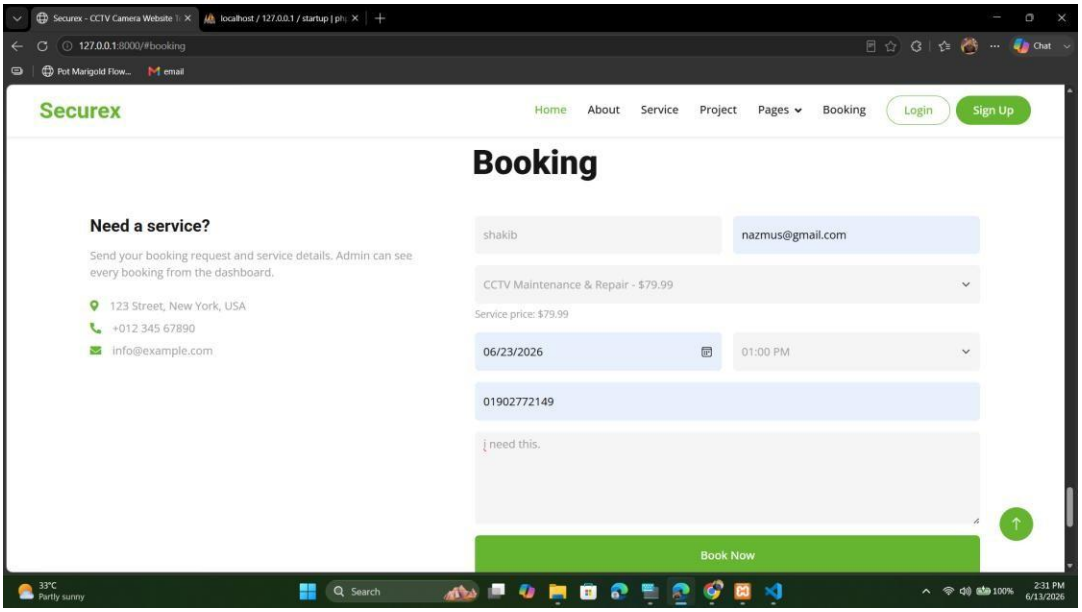


Fig. 3.3: Service Booking Form. An authenticated user enters contact information, selects a service, date, and time slot, adds an optional message, and submits the request for server-side validation.

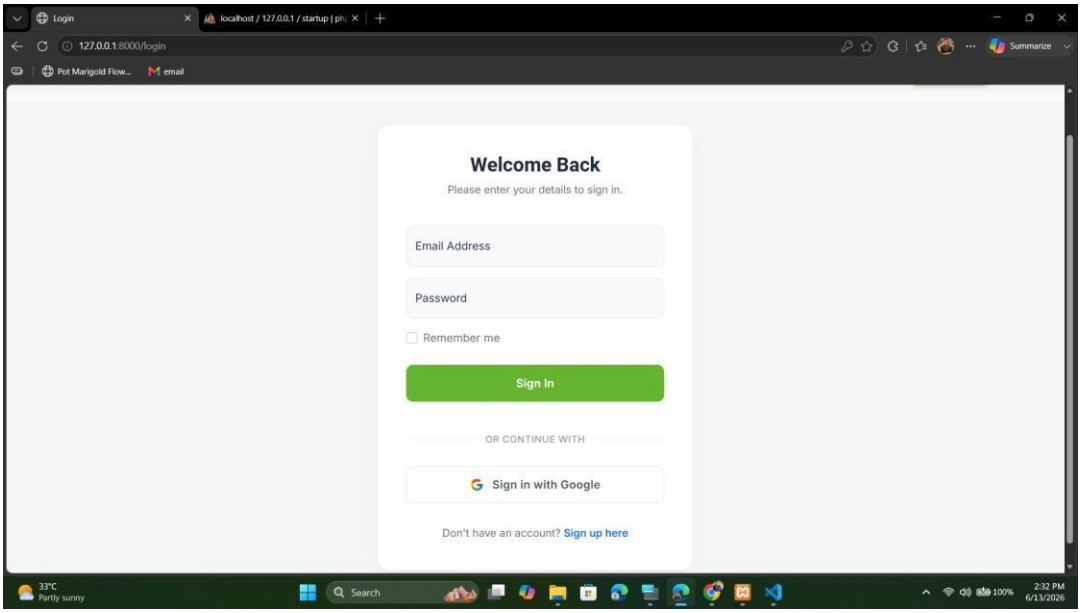


Fig. 3.4: Login Page. The page supports conventional email-and-password authentication and a Google OAuth sign-in option, while also providing account recovery and registration links.

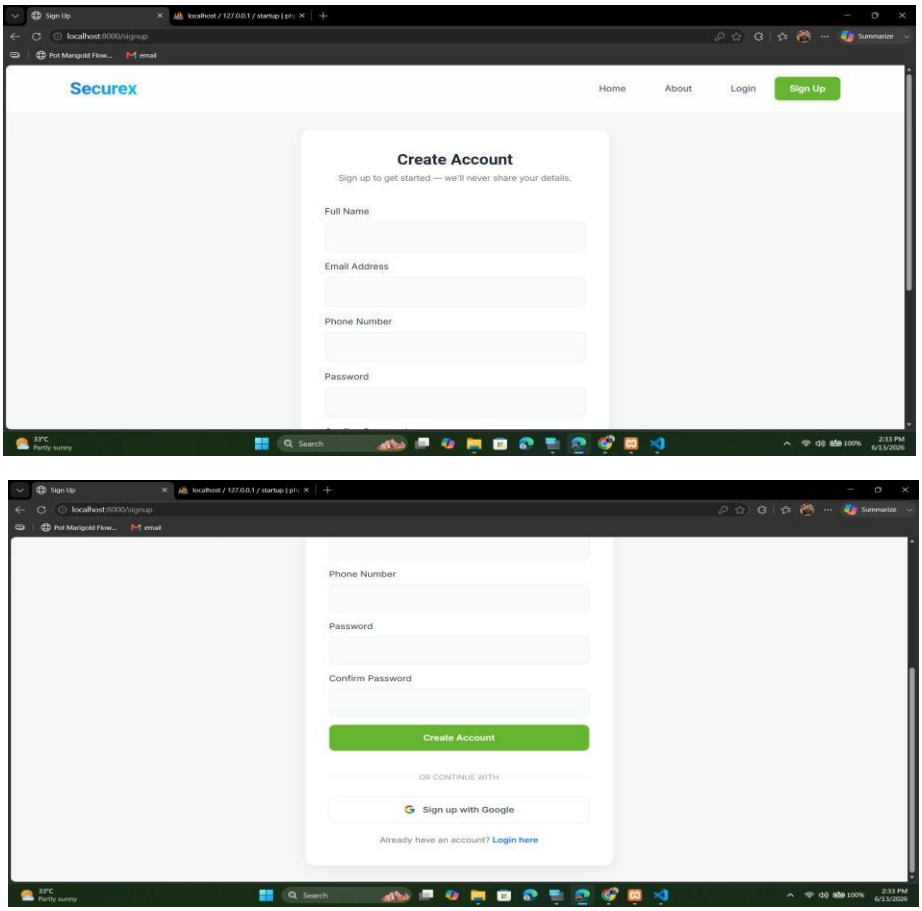


Fig. 3.5: User Registration Page shown in two parts. The upper view contains the name, email, and phone fields; the lower view contains password confirmation, the Create Account button, Google registration, and the link for existing users.

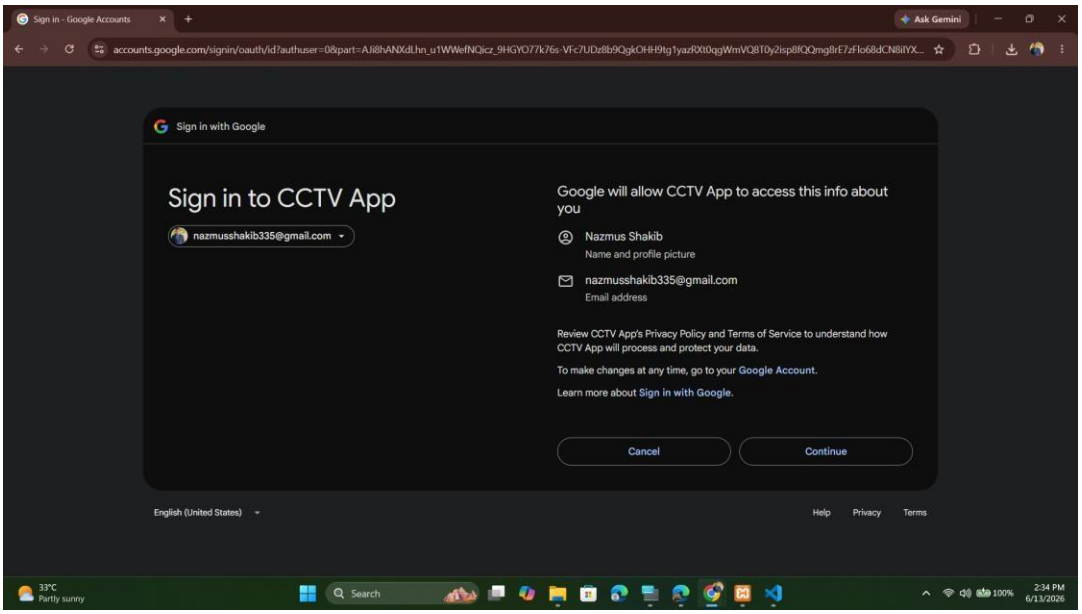


Fig. 3.6: Google OAuth 2.0 Consent Screen. It identifies the selected Google account, explains the profile and email access requested by Securex, and provides Cancel and Continue controls before authentication returns to the application.

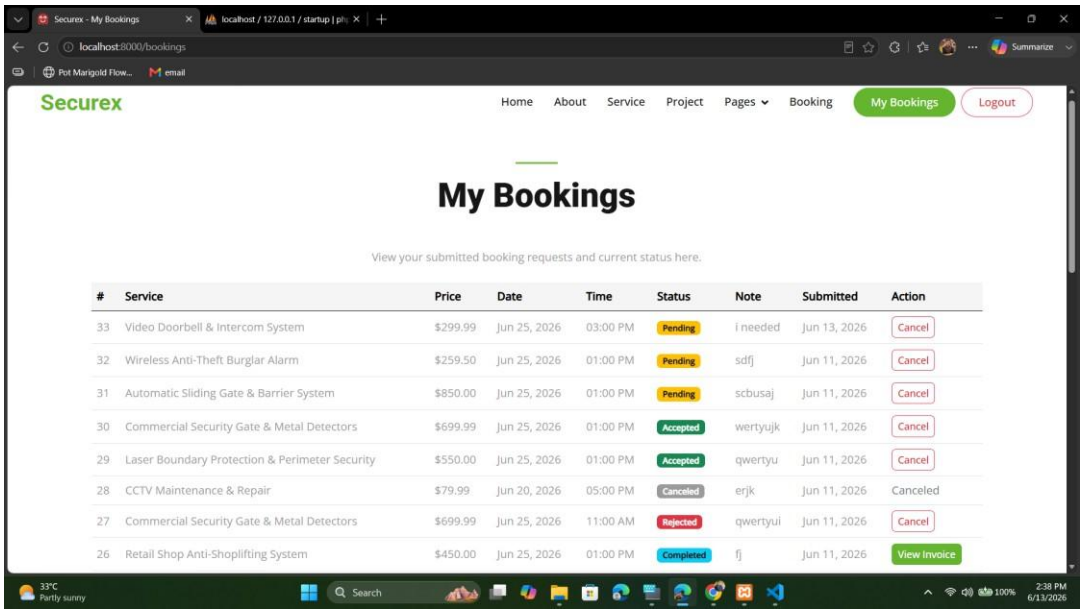


Fig. 3.7: My Bookings Page (User View). The table summarizes service, price, date, time, current status, note, and submission date, while the Action column displays cancellation or invoice controls only when the booking state permits them.

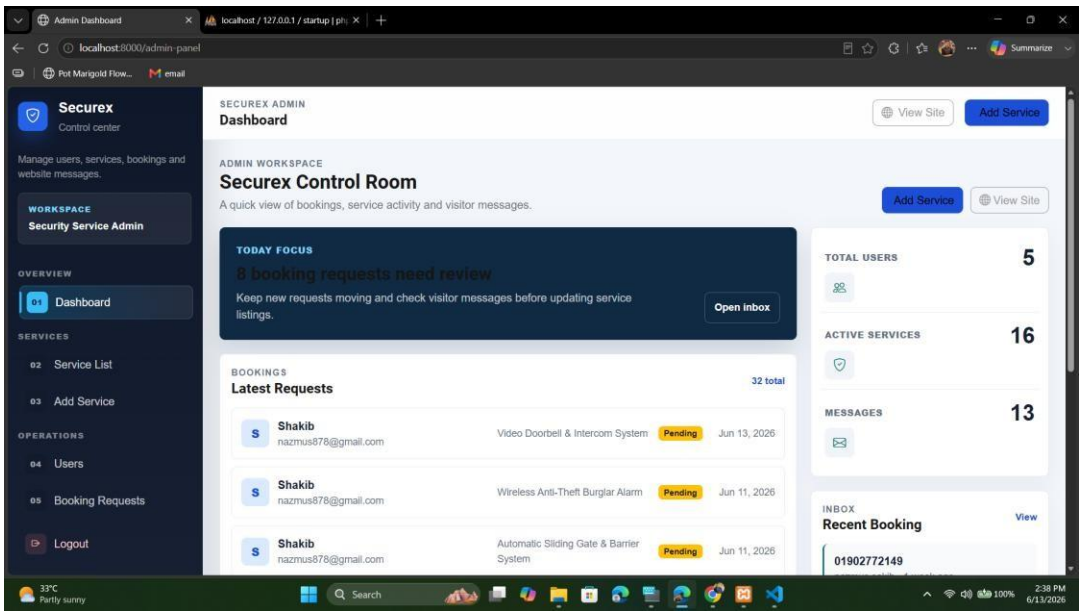


Fig. 3.8: Admin Dashboard - Securex Control Room. Summary cards show total users, active services, messages, and bookings; the focus panel highlights pending work; and the recent-bookings area gives the administrator a quick operational overview.

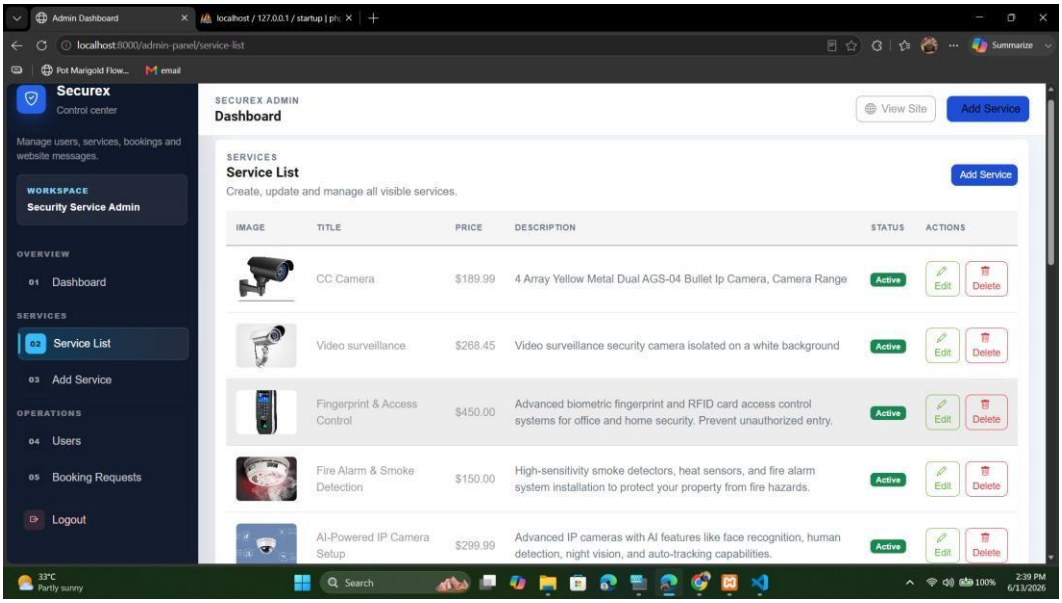


Fig. 3.9: Admin Service List Management Panel. The table displays service images, titles, prices, descriptions, visibility status, and maintenance actions so that the public catalog can be updated without editing application code.

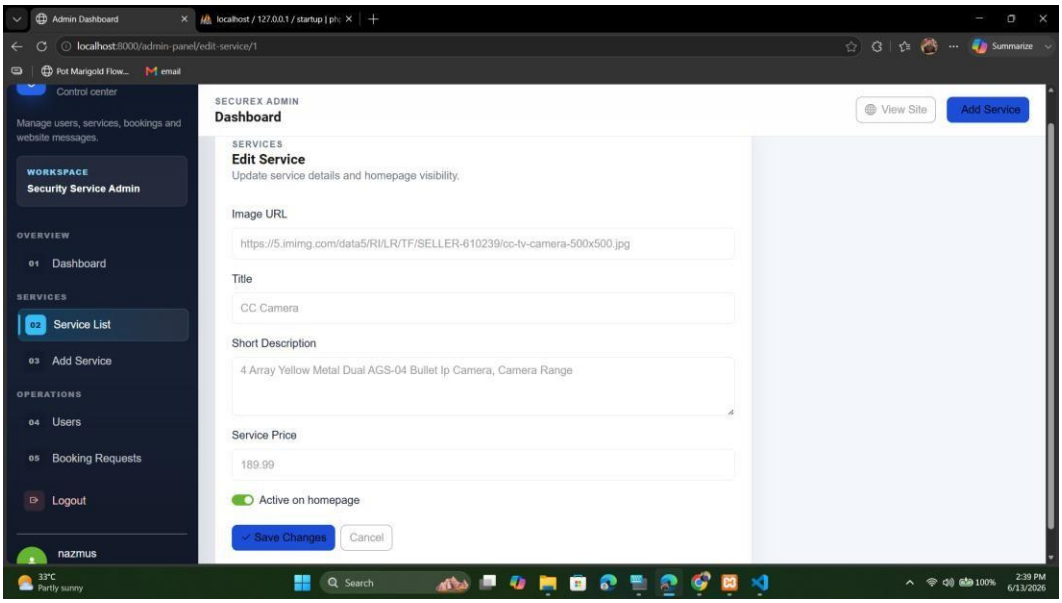


Fig. 3.10: Admin Edit Service Form. The administrator can update the image URL, title, short description, price, and homepage visibility of an existing service; saving the form updates the database-driven public interface.

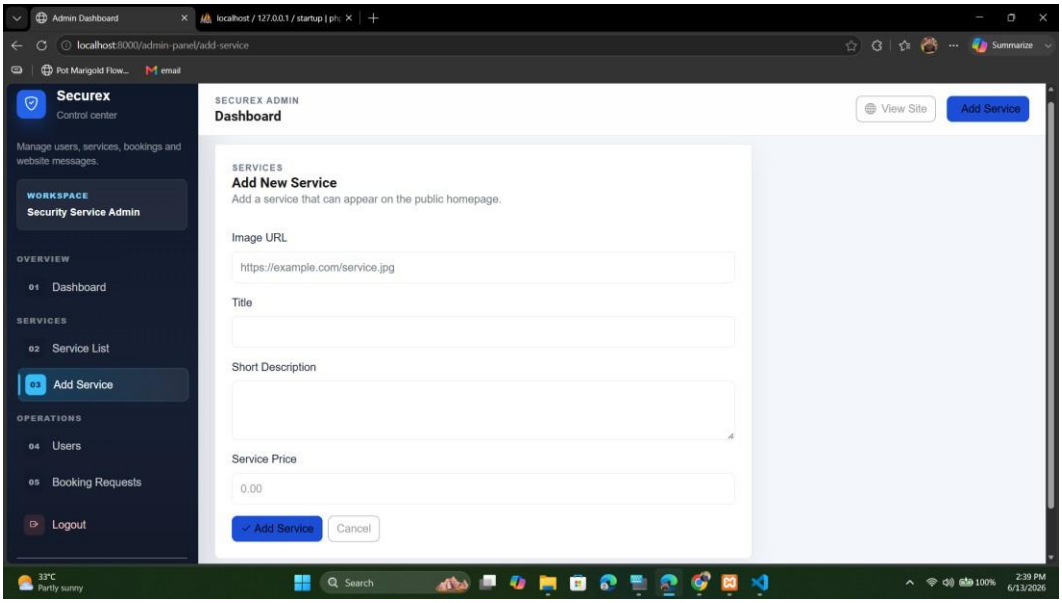


Fig. 3.11: Admin Add New Service Form. A new catalog entry is created by supplying its image URL, title, description, and price, after which the service becomes available for administration and optional public display.

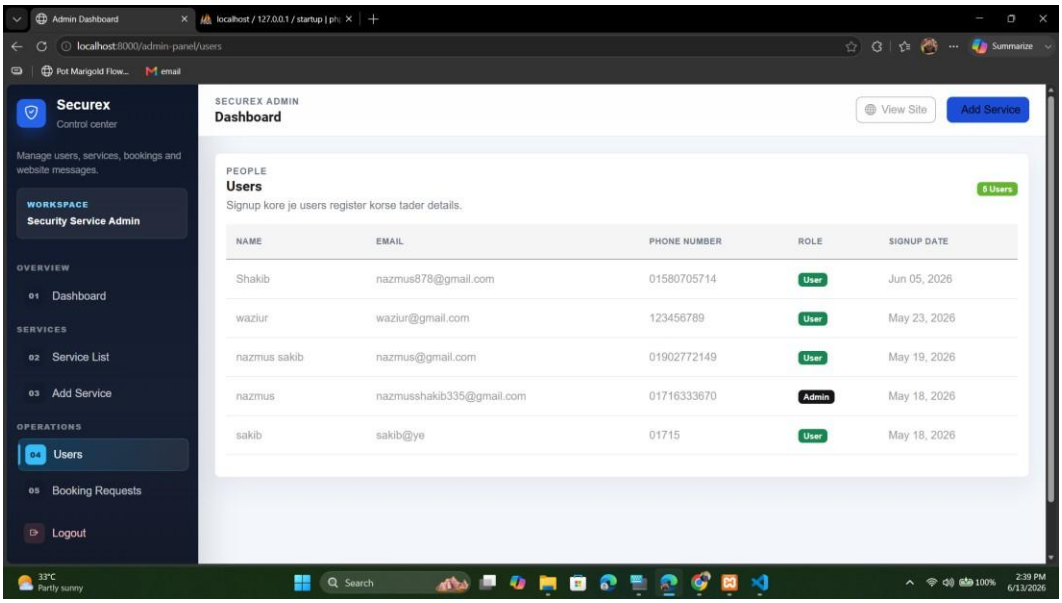


Fig. 3.12: Admin User Management Panel. The centralized directory lists each registered user's name, email, phone number, role, and registration date, helping the administrator distinguish customer and administrative accounts.

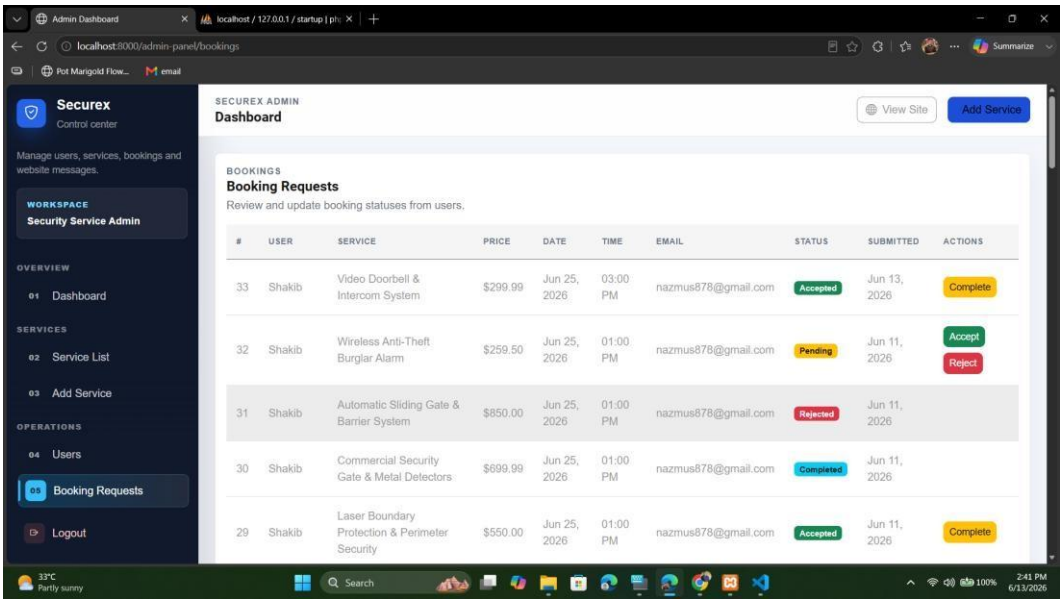


Fig. 3.13: Admin Booking Requests Panel. Each row shows the customer, selected service, price, scheduled date and time, email, status, and submission date; context-sensitive buttons allow Pending requests to be accepted or rejected and Accepted work to be completed.

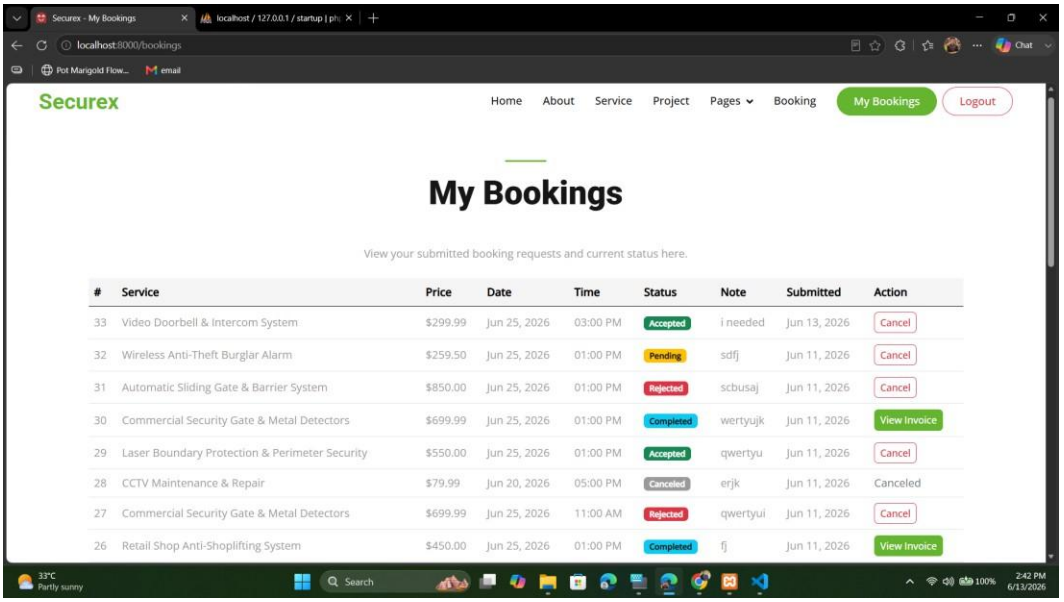


Fig. 3.14: My Bookings Status and Action Page. Customers can compare Pending, Accepted, Rejected, Canceled, and Completed requests, with cancellation and invoice actions displayed according to the rules of each status.

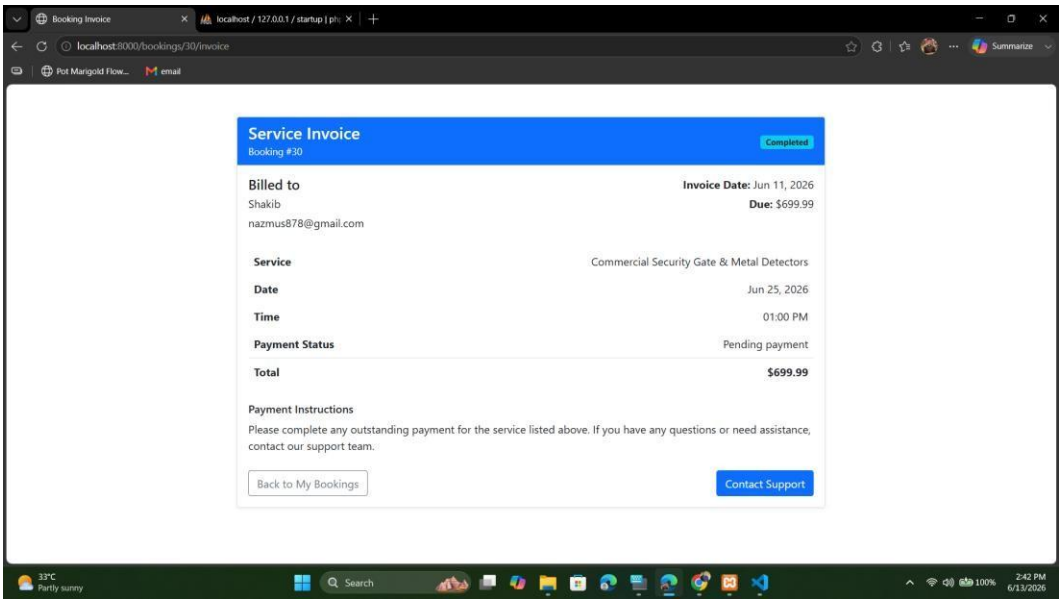


Fig. 3.15: Service Invoice View. The invoice presents the invoice and booking identifiers, customer details, selected service, appointment date and time, payment status, total amount, and the controls used to return to bookings or cancel an eligible request.

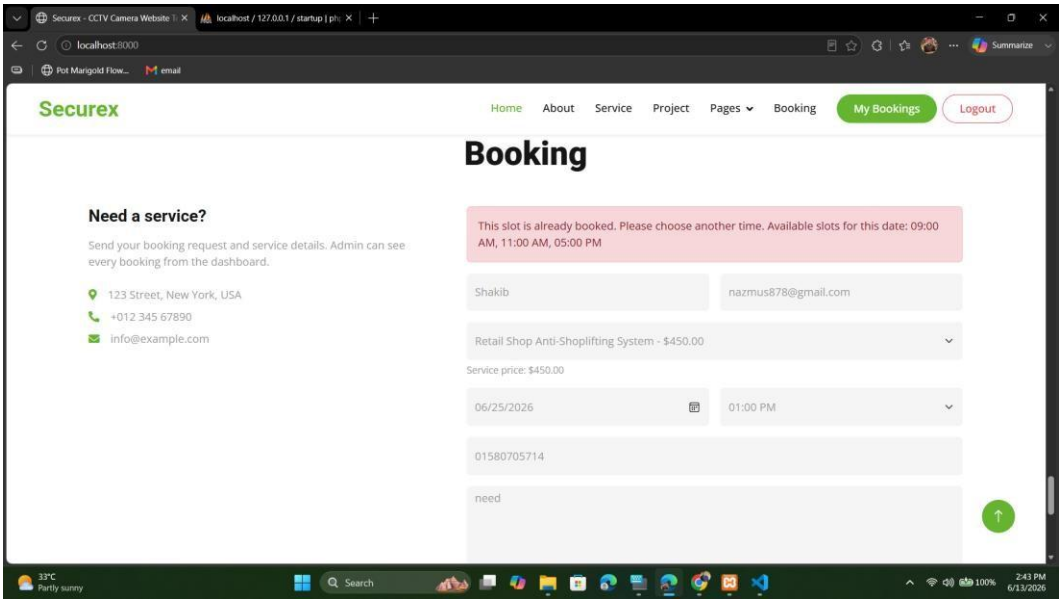


Fig. 3.16: Booking Time Slot Conflict Validation. The warning identifies that the selected service, date, and time are already occupied and displays alternative available slots, preventing duplicate bookings without forcing the customer to restart the form.

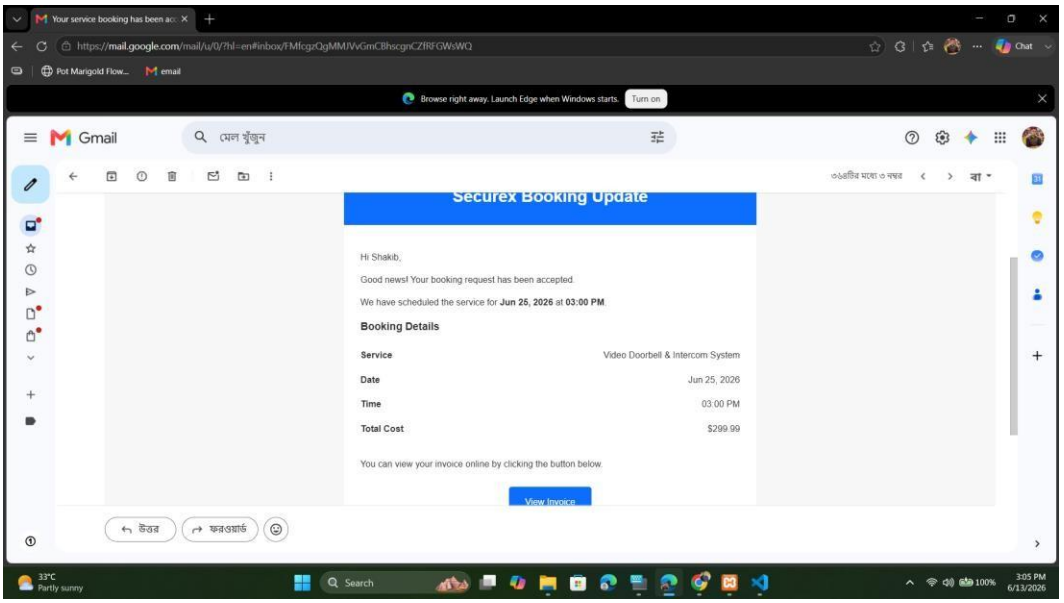


Fig. 3.17: Automated Booking Update Email. The message confirms the changed booking status and includes the service, confirmed date and time, total cost, and invoice access, giving the customer an immediate and traceable record of the administrator's action.

The results collectively demonstrate that the Securex platform operates as a coherent, integrated system. The user journey from discovering a service on the public portal, through authenticated booking submission, through administrative review and acceptance, to the receipt of an automated email notification with an invoice link was validated end to end. The intelligent time slot validation mechanism proved robust, successfully blocking all conflicting booking attempts during testing while consistently providing relevant alternative slot suggestions. The administrative interface provided a complete and efficient management layer, confirming the system's readiness for a pilot deployment context.

CHAPTER IV

Conclusions and Recommendations

4.1 Conclusions

This project report has documented the design, development, and successful implementation of Securex, a comprehensive web based CCTV and Security Service Management System built on the PHP Laravel framework. The system was conceived to address the operational inefficiencies and service quality limitations inherent in manual security service management processes, and the results confirm that all primary objectives have been achieved.

The platform successfully delivers a fully dynamic, database driven public facing website that accurately represents the company's service offerings. The dual mode authentication system combining traditional credential based login with Google OAuth 2.0 integration provides both security and convenience for end users. The core booking engine, with its intelligent time slot conflict detection and alternative slot suggestion mechanism, represents a significant practical improvement over conventional manual booking methods, eliminating the possibility of double bookings and providing a superior client experience.

The administrative dashboard successfully centralizes all operational management functions, enabling administrators to maintain the service catalog, manage user accounts, and process the entire booking lifecycle from a single, intuitive interface. The automated email notification and PDF invoice generation pipeline effectively closes the communication loop between the service provider and the client, enhancing transparency and professionalism.

On a developmental level, the project provided valuable hands on experience in applying the MVC architectural pattern in a real world context, integrating third party OAuth libraries, designing and querying relational databases with ORM, and diagnosing and resolving non trivial framework configuration issues such as the routing refactoring

challenge. The successful resolution of these challenges underscores the importance of a systematic, framework idiomatic approach to web application development.

In conclusion, Securex stands as a fully functional and extensible platform that demonstrates the practical application of modern web development principles to solve genuine business problems in the security services domain. It provides a strong foundation upon which further features and refinements can be built.

4.2 Recommendations for Future Work

While the current implementation of Securex fulfills its defined objectives, several avenues for future enhancement have been identified that would substantially increase the platform's commercial viability, operational scope, and user value:

1. Integrated Payment Gateway: The most impactful near term enhancement would be the integration of a payment gateway such as Stripe, PayPal, or a local solution (e.g., bKash or SSLCommerz). This would allow clients to make secure online payments at the time of booking or upon invoice receipt, transforming the platform from a booking management system into a complete transactional marketplace.

2. Live Technician Tracking: The integration of a real time GPS tracking module leveraging the Google Maps API or similar would allow clients to monitor the live location of their assigned service technician on the day of the appointment. This feature would dramatically improve client confidence and reduce the need for manual status inquiries.

3. In App Messaging and Support Chat: A real time communication channel between clients and the support team, built using Laravel WebSockets or a service like Pusher, would allow for instant query resolution and progress updates without requiring the client to leave the platform.

4. Service Ratings and Reviews: The introduction of a post service review and rating system would enable clients to provide feedback on completed bookings. This would create a valuable trust building mechanism for prospective customers and provide the service provider with actionable data for service quality improvement.

5. Advanced Analytics Dashboard: Expanding the admin dashboard with comprehensive business analytics including revenue trends over time, service popularity metrics, booking conversion rates, and peak demand analysis would

transform it into a powerful business intelligence tool to inform strategic decision making.

6. Mobile Application Development: The development of a companion mobile application for both Android and iOS (using Flutter or React Native) would significantly broaden the platform's accessibility, enabling clients to book services, track booking status, and receive push notifications on the go.

7. Multi-Tenant Architecture: Refactoring the application to support a multi tenant model would allow the platform to be offered as a Software as a Service (SaaS) product to multiple independent security service companies, each with their own isolated data, admin panel, and branding configuration.

References

[1] Laravel, “Laravel Documentation,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs. [Accessed: Jul. 22, 2026].

[2] Laravel, “Laravel Socialite,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/socialite. [Accessed: Jul. 22, 2026].

[3] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, Internet Engineering Task Force, Oct. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749. [Accessed: Jul. 22, 2026].

[4] R. T. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. [Accessed: Jul. 22, 2026].

[5] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[6] Oracle Corporation, “MySQL 8.0 Reference Manual,” *MySQL Documentation*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. [Accessed: Jul. 22, 2026].

[7] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: https://owasp.org/Top10/. [Accessed: Jul. 22, 2026].

[8] PHP Documentation Group, “PHP Manual,” *PHP Documentation*. [Online]. Available: https://www.php.net/manual/en/. [Accessed: Jul. 22, 2026].

[9] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993.

[10] Laravel, “Routing,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/routing. [Accessed: Jul. 25, 2026].

[11] Laravel, “Blade Templates,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/blade. [Accessed: Jul. 25, 2026].

[12] Laravel, “Eloquent ORM: Getting Started,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/eloquent. [Accessed: Jul. 25, 2026].

[13] Laravel, “Validation,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/validation. [Accessed: Jul. 25, 2026].

[14] Laravel, “Middleware,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/middleware. [Accessed: Jul. 25, 2026].

[15] Laravel, “Authentication,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/authentication. [Accessed: Jul. 25, 2026].

[16] Laravel, “Authorization,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/authorization. [Accessed: Jul. 25, 2026].

[17] Laravel, “Mail,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/mail. [Accessed: Jul. 25, 2026].

[18] Laravel, “Database: Getting Started,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/database. [Accessed: Jul. 25, 2026].

[19] Laravel, “HTTP Tests,” *Laravel Documentation*. [Online]. Available: https://laravel.com/docs/12.x/http-tests. [Accessed: Jul. 25, 2026].

[20] Google, “Using OAuth 2.0 for Web Server Applications,” *Google for Developers*. [Online]. Available: https://developers.google.com/identity/protocols/oauth2/web-server. [Accessed: Jul. 25, 2026].